Q12. Extracting email addresses and URLs from an image

Original question: Analyze a forensic disk image to extract network-related artifacts, including available email addresses and URLs. Examine the generated output files and record the relevant findings.

Aim

Extract candidate email and URL artifacts from a supplied disk image and examine their feature files and context.

Requirements

Verified disk image/segments, bulk_extractor, storage for output and an offline analysis workspace. No image has been supplied here. Reference: [R11].

Procedure

Step 1 - Prepare and check available scanners

```
mkdir -p "$HOME/EHF_Exam/Q12"
cd "$HOME/EHF_Exam/Q12"
IMAGE='<DISK_IMAGE_PATH>'
sha256sum "$IMAGE" | tee image_before.sha256
bulk_extractor -V
bulk_extractor -h > bulk_help.txt
sed -n '1,180p' bulk_help.txt
```

Read the installed help for enabled email/URL feature scanners and supported image formats. Use all necessary segments. If the build cannot read a container, use the lab-approved verified raw export, not a renamed E01 file. The source must remain unmodified.

Step 2 - Extract into a fresh directory

```
OUT="$PWD/bulk_q12_run1"
test ! -e "$OUT" && bulk_extractor -o "$OUT" "$IMAGE" \
  > bulk_run.log 2>&1
cat bulk_run.log
ls -l "$OUT"
```

Use a new OUT name for each run. If the directory already exists, the test intentionally prevents a rerun; choose run2 after reviewing prior results. `-o` names the output directory. Bulk extraction searches bytes and supported encoded/compressed content without requiring a filesystem walk; it produces candidates that need validation. Do not use an option that wipes an earlier output directory.

Step 3 - Inspect the generated files

```
for NAME in email.txt url.txt email_histogram.txt url_histogram.txt; do
  if [ -f "$OUT/$NAME" ]; then
    printf '\n--- %s ---\n' "$NAME"
    sed -n '1,40p' "$OUT/$NAME"
  else
    printf 'Not generated: %s\n' "$NAME"
  fi
done
sha256sum -c image_before.sha256
```

Typical feature files have comments followed by a forensic position, extracted feature and nearby context separated by tabs. Histogram files summarize occurrence counts; they do not establish distinct users or independent events. report.xml, when generated, records run details. Filenames/scanners vary: inspect

actual output and logs before concluding an artifact is absent.

Step 4 - Validate selected candidates

Choose a relevant email and URL only if present. Retain each exact feature line, forensic position, context and source image hash. Cross-reference with Autopsy or TSK when a containing file can be identified; record that file path and metadata identifier. A simple position may be a raw byte offset; a recursive encoded/decompressed position is a forensic path, not a directly seekable disk offset. Do not treat all positions as plain numbers.

Expected output - illustrative feature lines

```
# Simplified tab-separated examples; not actual findings
123456  student@example.org        nearby printable context
124000  https://portal.example.org/  nearby printable context
```

A file may contain zero relevant feature lines. Comment headers alone do not mean an address was discovered. Results can include software documentation, templates or stale deleted content.

Interpretation, observation and result

Record type, literal value, feature file, position, context, containing file if resolved, and confidence. An email string does not prove a message was sent; a URL does not prove it was visited. Explain which context supports your interpretation and what remains unresolved. The result is an auditable list of candidates and validated context, or a documented absence under the scanners used.

Common errors

- Output exists: choose a fresh directory; preserve the original results.
- Empty/missing feature file: check enabled scanners, run completion and format; report "not found by this run" if appropriate.
- Process killed or disk full: preserve logs, check df -h and memory; incomplete output is not a complete examination.

Viva questions

- Does bulk_extractor need to mount the filesystem? No; it scans image content directly.
- What is a feature file? A list of candidate features with positions and context.
- Does a URL string prove browsing? No; application history or other context is needed.
- Why keep positions? They allow a candidate to be traced back for validation.